

```

options(OutDec = ".") # Asegurar punto decimal en este chunk
# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorización de parámetros principales
altura_inicial <- sample(10:20, 1)
pendiente <- round(runif(1, -0.7, -0.3), 2)
tiempo_final <- round(-altura_inicial / pendiente)
tiempo_pregunta <- sample(seq(2, tiempo_final - 2, by=2), 1)
altura_correcta <- altura_inicial + pendiente * tiempo_pregunta

# Aleatorización contextual
unidades_altura <- c("Altura (m)", "Nivel (m)", "Distancia vertical (m)", "Profundidad (m)")
unidad_altura <- sample(unidades_altura, 1)

unidades_tiempo <- c("Tiempo (s)", "Duración (segundos)", "Periodo (s)", "Intervalo (segundos)")
unidad_tiempo <- sample(unidades_tiempo, 1)

contextos <- c("esquiador descendiendo", "avión aterrizando", "submarino sumergiéndose", "cohetes ascendiendo")
contexto <- sample(contextos, 1)

# Generación de opciones de respuesta
opciones <- c(round(altura_correcta),
              round(altura_correcta + 2),
              round(altura_correcta - 2),
              sample(c(altura_inicial, 0), 1))

# Aleatorizar orden usando sample() en lugar de shuffle
opciones <- sample(opciones)

indice_correcto <- which(opciones == round(altura_correcta))
solucion <- integer(4)
solucion[indice_correcto] <- 1

options(OutDec = ".") # Asegurar punto decimal en este chunk
# Código Python para generar gráficas
codigo_base_python <- "
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('default')
altura_inicial = %s
pendiente = %.2f
tiempo_final = %s
tiempo_pregunta = %s
altura_correcta = %.2f
"

# Reemplazar valores en el código Python
codigo_python_base <- sprintf(codigo_base_python,
                              altura_inicial,
                              pendiente,
                              tiempo_final,
                              tiempo_pregunta,
                              altura_correcta)

```

```

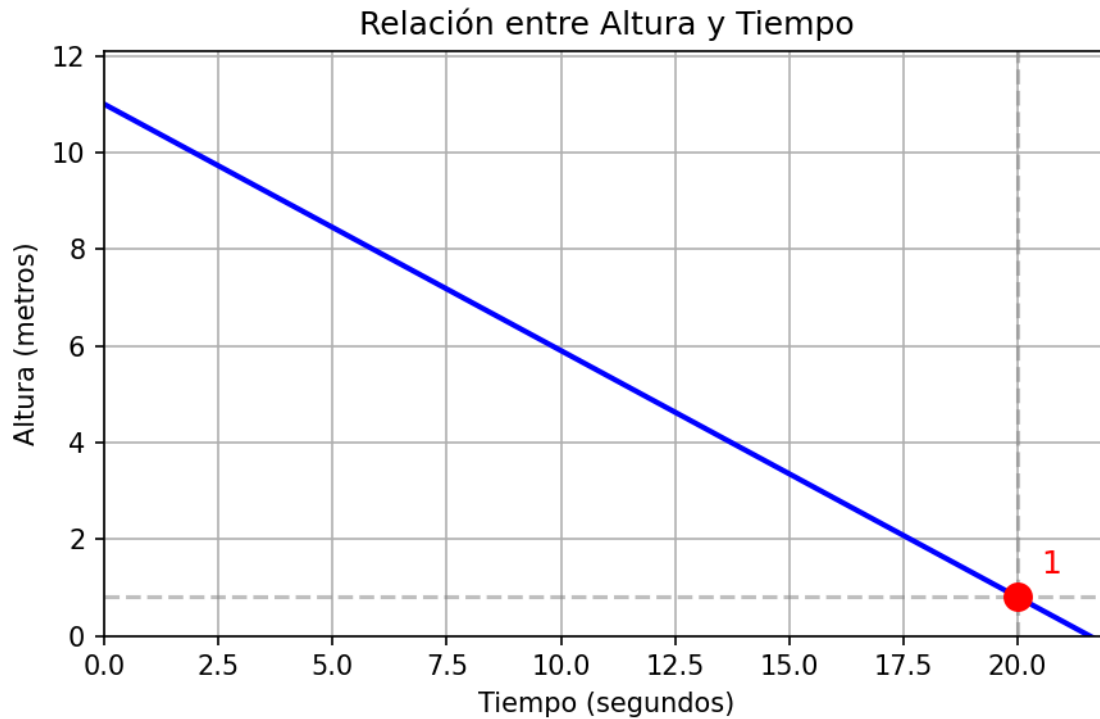
# Código para graficar la situación física
codigo_python_grafica <- paste0(codigo_python_base, "
plt.figure(figsize=(6, 4))
tiempos = np.linspace(0, tiempo_final, 100)
alturas = altura_inicial + pendiente * tiempos
plt.plot(tiempos, alturas, color='blue', linewidth=2)
plt.scatter(tiempo_pregunta, altura_correcta, color='red', s=100, zorder=5)
plt.axhline(altura_correcta, color='gray', linestyle='--', alpha=0.5)
plt.axvline(tiempo_pregunta, color='gray', linestyle='--', alpha=0.5)
plt.annotate(f'{round(altura_correcta)}', (tiempo_pregunta + 0.5, altura_correcta + 0.5),
            fontsize=12, color='red')
plt.title('Relación entre Altura y Tiempo')
plt.xlabel('Tiempo (segundos)')
plt.ylabel('Altura (metros)')
plt.grid(True)
plt.xlim(0, tiempo_final)
plt.ylim(0, max(alturas)*1.1)
plt.tight_layout()
plt.savefig('grafica.png', dpi=150)
plt.close()
")

# Ejecutar el código Python para generar la gráfica
py_run_string(codigo_python_grafica)

```

Question

La gráfica muestra la relación entre Nivel (m) y Tiempo (s) de un objeto en movimiento durante su trayectoria de esquiador descendiendo.



¿Cuál es la Nivel del objeto a los 20?

Answerlist

- 3
- -1
- 11
- 1

Solution

La altura se calcula mediante la ecuación lineal:

$$\text{Altura} = 11 + (-0.51) \times \text{Tiempo}$$

Para 20 segundos:

$$\text{Altura} = 11 + (-0.51) \times 20 = 1 \text{ metros}$$

La gráfica muestra claramente este valor en el punto marcado como referencia.

Answerlist

- Falso
- Falso
- Falso
- Verdadero

Meta-information

exname: funciones_lineales_interpretacion_grafica_v1 extype: schoice exsolution: 0001 exshuffle: TRUE
exsection: Interpretación de gráficas